OSPilot: Offline Desktop AI Assistant

Complete Beginner's Guide & Interview Handbook

OSPilot is a production-quality, 100% local, offline AI Desktop Assistant engineered with an Electron frontend, Python FastAPI backend, local Ollama LLMs (*Qwen3 / Gemma3 / Qwen2.5-Coder*), FAISS Vector RAG pipeline, LangGraph Multi-Agent Architecture, AI Coding Assistant, Desktop/Browser Automation, and a Multi-Tier Memory System. Zero cloud dependencies.

■ **What is OSPilot?** An offline AI assistant running on your PC that controls apps, searches local files, writes code, and remembers past interactions.	■ **Technology Stack** Electron (Frontend UI), FastAPI (Python Backend), Local Ollama LLM, FAISS & SQLite (RAG Engine).
■■ **Zero Cloud Privacy** All processing stays on your device. No user data, files, or prompt text ever leave your computer.	■ **Action Safety** Destructive commands (deleting files, shutting down, closing apps) require interactive user popup confirmation.

Handbook Navigation / Table of Contents

1. Project Overview & Elevator Pitch

■■ How to explain OSPilot in an interview (Elevator Pitch):

'OSPilot is an open-source, production-ready desktop AI assistant built with Electron and Python FastAPI. It runs entirely offline on the user's computer using local open-weight LLMs via Ollama. Unlike simple chatbots, OSPilot features a LangGraph multi-agent orchestrator that routes user requests to specialized nodes for file search (FAISS vector database), document Q&A; (RAG), OS desktop control (PyAutoGUI), automated web browser search (Playwright), and AI code generation. It also implements token-by-token real-time streaming over SSE and a 4-tier memory system.'

Key Value Propositions

- **100% Data Privacy & Security:** Zero API keys required for core features. Ideal for enterprise codebases and confidential docs.
- **Sub-Second Real-Time UI:** Uses Server-Sent Events (SSE) for instant token streaming without visual lag.
- **Resilient Offline Execution:** Automatically falls back across local LLM models (`qwen3:8b` -> `gemma3:latest` -> `qwen2.5-coder:7b`).
- **Human-in-the-Loop Safety:** Dangerous computer commands (deleting files, shutting down) are trapped until the user confirms via dialog popup.

2. Technology Stack Breakdown

Layer	Technology	Role & Purpose
Frontend Desktop UI	Electron + JavaScript + HTML5/CSS3	Renders desktop window, sidebar navigation, real-time SSE stream reader, and safety confirmation modals.
Backend API Engine	Python 3.10+ & FastAPI	High-performance asynchronous REST API handling endpoints, background tasks, and business services.
Local LLM Server	Ollama (`qwen3`, `gemma3`, `qwen2.5-coder`)	Serves local GGUF open-weights models with VRAM management and HTTP inference endpoint.
Multi-Agent Framework	LangGraph (`StateGraph`)	Orchestrates autonomous agent graphs: Planner, Retriever, Automation, Coding, & Memory nodes.
Vector Search (RAG)	FAISS + `nomic-embed-text`	Generates 768-dim vector embeddings and performs ultra-fast L2 similarity search over local documents.
Metadata & Memory DB	SQLite (Write-Ahead Logging WAL Mode)	Stores conversation history, file chunk references, explicit remembered facts, and settings.
OS & Web Automation	PyAutoGUI & Playwright	Simulates keyboard/mouse inputs for desktop control and executes automated browser searches.

3. Directory Structure & File Map

Understand where every single piece of logic resides in the workspace:

File / Folder Path	What it does (In simple words)
`electron/main.js`	Electron root process. Creates window, spawns Python FastAPI subprocess automatically, handles IPC.
`frontend/index.html`	7-page single-page application (Chat, RAG, Coding, Automation, Memory, Settings, Logs).
`frontend/renderer.js`	Frontend logic. Handles UI tabs, fetch requests, theme toggles, SSE stream parsing.
`backend/app/main.py`	FastAPI entrypoint. Registers router, CORS middleware, global error handlers.
`backend/app/api/v1/router.py`	Central router aggregating chat, search, rag, automation, browser, coding, agent, memory routes.
`backend/app/core/config.py`	Application configuration (Ollama URL, model defaults, DB paths).
`backend/app/db/session.py`	SQLite database engine hardened with PRAGMA journal_mode=WAL for 10x faster concurrent access.
`backend/app/services/ollama_service.py`	Manages HTTP connection to Ollama LLM with intelligent model fallback retry chain.
`backend/app/services/multi_agent_service.py`	LangGraph StateGraph workflow organizing Planner, Retriever, Coding, Automation, & Memory nodes.
`backend/app/services/rag_service.py`	RAG pipeline. Reads PDF/DOCX/TXT/MD, chunks text, generates embeddings, stores in FAISS.
`backend/app/services/desktop_automation_service.py`	Controls OS desktop (launching apps, volume, screenshots, file operations). Requires safety confirmation.
`backend/app/services/memory_system_service.py`	Manages 4 memory tiers: Short-term cache, Long-term vector facts, History, & User Preferences.

4. Core Subsystem Explanations

A. RAG (Retrieval-Augmented Generation) Pipeline

What is RAG? Standard LLMs don't know your private personal files. RAG injects relevant text snippets from your files into the LLM prompt.
1. Document Loading: document_loader.py extracts plain text from PDF, DOCX, TXT, and Markdown files.
2. Text Chunking: Large text is broken into 500-character chunks with 50-character overlaps.
3. Vector Embeddings: embedding_service.py converts each chunk into a 768-dimensional numerical vector using nomic-embed-text.
4. FAISS Indexing: faiss_service.py saves vectors in FAISS index for ultra-fast L2 distance similarity matching.
5. Retrieval Q&A;: When you ask a question, your query is embedded, top 5 matching chunks are fetched, and passed to Ollama to answer.

B. LangGraph Multi-Agent Architecture

Instead of asking one LLM to do everything, OSPilot breaks complex queries into a graph of specialized nodes:
• **Planner Node:** Decides user intent and selects target worker agents.
• **Retriever Node:** Searches local documents and vector memory.
• **Coding Node:** Explains, debugs, or writes code modules.
• **Automation Node:** Prepares PyAutoGUI or Playwright actions.
• **Synthesizer Node:** Combines outputs into a unified answer and handles error retries.

C. Real-Time Token Streaming (SSE)

Server-Sent Events (SSE): Fast HTTP streaming endpoint (POST /api/v1/chat/stream). FastAPI yields data: {"token": "Hello"} chunks. The Electron UI uses JavaScript's fetch() with ReadableStream reader to display tokens word-by-word instantly.

D. Multi-Tier Memory Architecture

1. **Short-Term Memory:** Active conversation window (LRU cache).
2. **Long-Term Fact Store:** Explicit user facts stored in SQLite & FAISS vector memory.
3. **Conversation History:** Searchable database of past dialogue turns.
4. **User Preferences:** Key-value settings stored in SQLite.

5. Step-by-Step Execution Data Flow

What happens under the hood when a user types a prompt in OSPilot?

Step 1: UI Input	User types a prompt in Electron frontend and clicks 'Send' or presses Enter.
Step 2: HTTP Request	renderer.js sends an HTTP POST request to FastAPI backend (http://127.0.0.1:8000/api/v1/chat/stream).
Step 3: Route Handling	chat.py receives request, extracts prompt, and loads session memory context.
Step 4: LLM Connection	ollama_service.py formats prompt with system rules and sends stream request to Ollama on port 11434.
Step 5: SSE Streaming	FastAPI yields SSE data stream chunks. renderer.js decodes stream bytes with TextDecoder and appends text to DOM token-by-token.
Step 6: History Persistence	Once streaming finishes, full response is saved into SQLite ConversationHistory table.

6. Top 12 Interview Questions & Master Answers

Q1: What is OSPilot, and why did you build it as an offline desktop application?

OSPilot is a 100% offline, privacy-first AI desktop assistant. It was built to solve corporate data exfiltration risks when developers and users handle proprietary code, financial documents, or personal data. By serving models locally via Ollama and using Electron + FastAPI, zero user data leaves the machine.

Q2: Why did you choose FastAPI for the backend instead of Node.js express?

FastAPI provides native asynchronous Python support (`async/await`), automatic OpenAPI documentation, high-performance SSE streaming, and direct integration with Python AI ecosystems like LangGraph, FAISS, PyAutoGUI, and Playwright without needing inter-process bridges.

Q3: How does real-time streaming work between FastAPI and Electron?

We implemented a token streaming endpoint using FastAPI's `StreamingResponse` yielding Server-Sent Events (SSE). On the frontend, `renderer.js` uses JavaScript's `fetch()` API with `ReadableStream` reader to decode byte streams and update the UI live token-by-token.

Q4: How do you handle cold-start latency when Ollama loads LLM models into VRAM?

Cold LLM models take 5-10 seconds to load into GPU VRAM. We configured 120-second HTTP client timeouts in `OllamaService` and implemented a fallback model chain (`qwen3:8b` -> `gemma3:latest` -> fallback synthesis) to ensure high system resilience.

Q5: How does the RAG vector indexing pipeline work in OSPilot?

Documents (PDF, DOCX, TXT, MD) are extracted via `document_loader.py`, split into 500-character chunks with 50-character overlap, converted into 768-dim embeddings via `nomic-embed-text`, and stored in FAISS vector index. SQLite stores text snippets and metadata mapped 1:1 to FAISS vector IDs.

Q6: How did you optimize SQLite for high concurrent performance in a desktop app?

We enabled Write-Ahead Logging (`PRAGMA journal_mode=WAL;` and `PRAGMA synchronous=NORMAL;`) in `session.py`. This allows concurrent readers while a writer writes, eliminating database lock errors during streaming chat.

Q7: How is desktop automation safety enforced for dangerous OS actions?

Potentially destructive commands (deleting files, app termination, system shutdown/restart/sleep) require a `confirmed: true` flag. If false, the service returns `confirmation_required`. Electron catches this and displays an interactive modal dialog requiring explicit user approval.

Q8: What is LangGraph and how is it used in OSPilot's Multi-Agent system?

LangGraph is a framework for building stateful multi-agent workflows using graph nodes. In OSPilot, `multi_agent_service.py` defines a `StateGraph` with a Planner Node routing tasks to Retriever, Coding, Automation, and Memory nodes, ending at a Synthesizer node for error recovery.

Q9: How does the AI Coding Assistant analyze and debug code?

The Coding Assistant (`coding_assistant_service.py`) uses specialized system prompts tailored for `qwen2.5-coder:7b`. It can scan workspace project trees, explain logic, debug stack trace errors, refactor code, and generate docstrings.

Q10: What is the 4-tier memory system in OSPilot?

It consists of: 1) Short-Term Memory (active dialogue session cache), 2) Long-Term Memory (fact store saved in SQLite & FAISS vector space), 3) Conversation History (searchable database of past dialogue turns), and 4) User Preferences (key-value UI settings).

Q11: How does browser automation work in OSPilot?

It uses Playwright (`browser_automation_service.py`) to launch headless or headed browser instances to execute web searches, navigate pages, extract text, click buttons, and fill web forms automatically.

Q12: How do you package and deploy OSPilot for non-technical end-users?

OSPilot uses `electron-builder` (`package.json`) to bundle the Electron frontend and Python environment into standalone Windows installers (`.exe` / NSIS) and portable executables.

7. Quick Setup & Troubleshooting Cheat Sheet

Essential Terminal Commands

Action / Goal	Terminal Command
1. Navigate to Project	`cd "D:\OS Pilot"`
2. Activate Python Virtual Env	`.\venv\Scripts\Activate.ps1`
3. Install Python Dependencies	`pip install -r backend/requirements.txt`
4. Start Local Ollama Server	`ollama serve`
5. Pull Required Local LLM Model	`ollama pull qwen3:8b`
6. Run Backend Tests	`venv\Scripts\python.exe -m pytest backend/tests`
7. Launch Desktop Application	`npm start`

Common Interview Troubleshooting Scenarios

Symptom / Error	Root Cause	Quick Solution
Ollama Connection Refused	Ollama service is not running on localhost:11434.	Run `ollama serve` in terminal.
HTTP 408 / Large Git Push Error	Committed heavy node_modules or large FAISS binary files to Git.	Add `node_modules/` and `*.bin` to `.gitignore` and run `git rm --cached`.
SQLite Database Locked Error	Multiple database connection handles opening simultaneously.	Ensure SQLite WAL mode `PRAGMA journal_mode=WAL;` is enabled in `session.py`.
Playwright Browser Launch Crash	Chromium binaries are missing.	Run `playwright install chromium` inside virtual environment.